



POLITECNICO
MILANO 1863

MARSH training

Table of Contents

01

[Logistics](#)
[Documentation](#)
[Software to follow along](#)

02

[Introduction](#)
[Problem statement](#)
[Objectives](#)
[Standard selection](#)

03

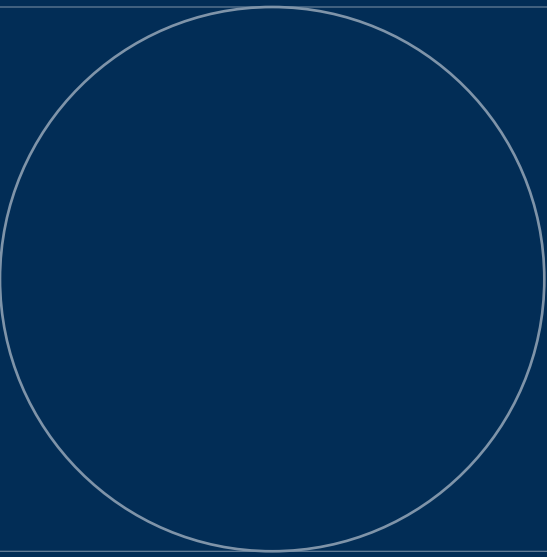
[Framework](#)
[Overall design](#)
[MAVLink protocol](#)
[MARSH](#)
[Node registration with Manager](#)
[Common features](#)
[Timing](#)

04

[Simulator nodes](#)
[Cutting up the helicopter](#)
[Controls](#)
[Flight models](#)
[Instruments](#)
[Visualisations](#)
[Motion cue](#)
[Extra tools](#)

05

[Development](#)
[New sim nodes](#)
[Exchanging data](#)
[Python](#)
[MATLAB](#)
[C and C++](#)
[Simulink](#)



Logistics

01

Documentation

- Main source of information: <https://marsh-sim.github.io>
 - Publicly available, automatically built from Git repository
 - Public repositories have only basic READMEs, content moved here
 - Includes automatically updated description of messages
 - Also has a list of [related academic publications](#)
- FRAME-Wiki <https://gitlab.polimi.it/DAER/frame-sim/frame-wiki/-/wikis/Home>
 - Laboratory-specific information and non-public flight models
 - Can be accessed from lab computers without authentication
- RPC-LH <https://gitlab.polimi.it/DAER/RPC/rpc-lh>
 - Project-specific information, most detailed description of motion platform
- Marek's thesis (to be published)

Software to follow along

One or more toolchains:

- Python from python.org or [uv](https://uv.rs)
- MATLAB with Simulink and some toolboxes (next page)
- C/C++ compiler (on Windows, Clang has [a simpler setup](#) than MSVC from Visual Studio)
- Godot Engine 4.4 (any patch like 4.4.1 is also OK)

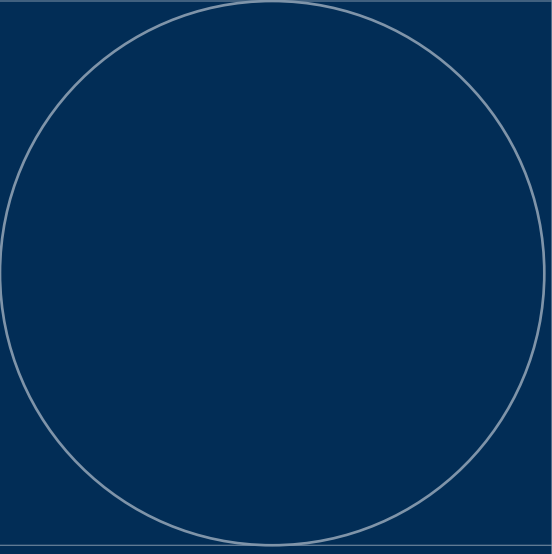
Optional generic tools:

- Git version control system
- Visual Studio Code (or another text editor)

Software to follow along

MATLAB component list, order like in the installer:

- Simulink
- Aerospace Blockset
- Aerospace Toolbox
- Control System Toolbox (state-space `ss`)
- DSP System Toolbox (some AFCS filters)
- Instrument Control Toolbox (Send UDP)
- Navigation Toolbox (`eu12rotm`, yes, for real)
- Optimization Toolbox (`fsolve`)
- Signal Processing Toolbox
- Simulink 3D Animation (somewhere in models)
- Simulink Real-Time (change endianness)
- Statistics and Machine Learning Toolbox (range, this also needs a toolbox)
- Symbolic Math Toolbox
- UAV Toolbox (**MAVLink**)



Introduction

02

Problem statement

We want tool(s) to experiment with helicopter flight simulation.

This chapter is a summary of previous material:

- Proposed options for the simulator architecture
- Simulator architecture comparison
- Standard selection for simulator integration

Available at <https://marsh-sim.github.io/rationale/>

Objectives

The stakeholders and their needs have been identified in the user stories format.

High level goals

As a researcher **I can** add new flight models, measurement devices and cueing systems, **so that** the simulator is useful for my research.

As a project leader **I can** use the simulator for commercial purposes without paying for any licenses, **so that** I can involve collaborators from industry environment.

As a project leader **I can** use the simulator without relying on any external service **so that** I am sure the simulator will work in the future, regardless of an external company

As a professor **I can** easily introduce students to the facility, **so that** they do practical projects.

As a student **I can** use widespread solutions, standards and libraries, **so that** I get practical experience for my career after graduating.

Objectives

Conducting studies

As a publication or thesis author **I can** easily gather all trial data into a single entity, **so that** they can be analyzed and presented in a written work.

As a user conducting trials **I can** control the whole simulator on my own using a single application, **so that** there are less people to schedule for a trial with a test subject, and I can iterate on my own.

As a human factors researcher **I can** see simulated view with an imperceptible delay, **so that** a human-in-the-loop piloting is viable.

As a human factors researcher **I can** use the motion platform in closed loop mode, **so that** the simulation realism for the pilot is increased.

Objectives

Compatibility

As a RPC project participant **I can** use Simulink and MBDyn models, **so that** I can reuse work already done in the project.

As a human factors researcher **I can** connect the simulation infrastructure to other flight simulation software eg. FlightGear or X-Plane, **so that** off-the-shelf visual models of cockpit and aircraft can be used.

As a user working on UAV support for HEMS missions (eg. MUM-T) **I can** communicate multiple aircraft (including unmanned) simultaneously, **so that** I can run shared simulations.

As a user of UAVs **I can** connect PX4 simulation, **so that** I can collaborate with the drone lab and industrial partners.

As a user of G-Seat **I can** connect the same simulator to moving platform and other cueing devices, **so that** a comparative study can be performed

Objectives

Developer experience

As a simulator developer **I can** reuse common elements in different configurations, **so that** there is less work repeated to prepare the simulator for a new study.

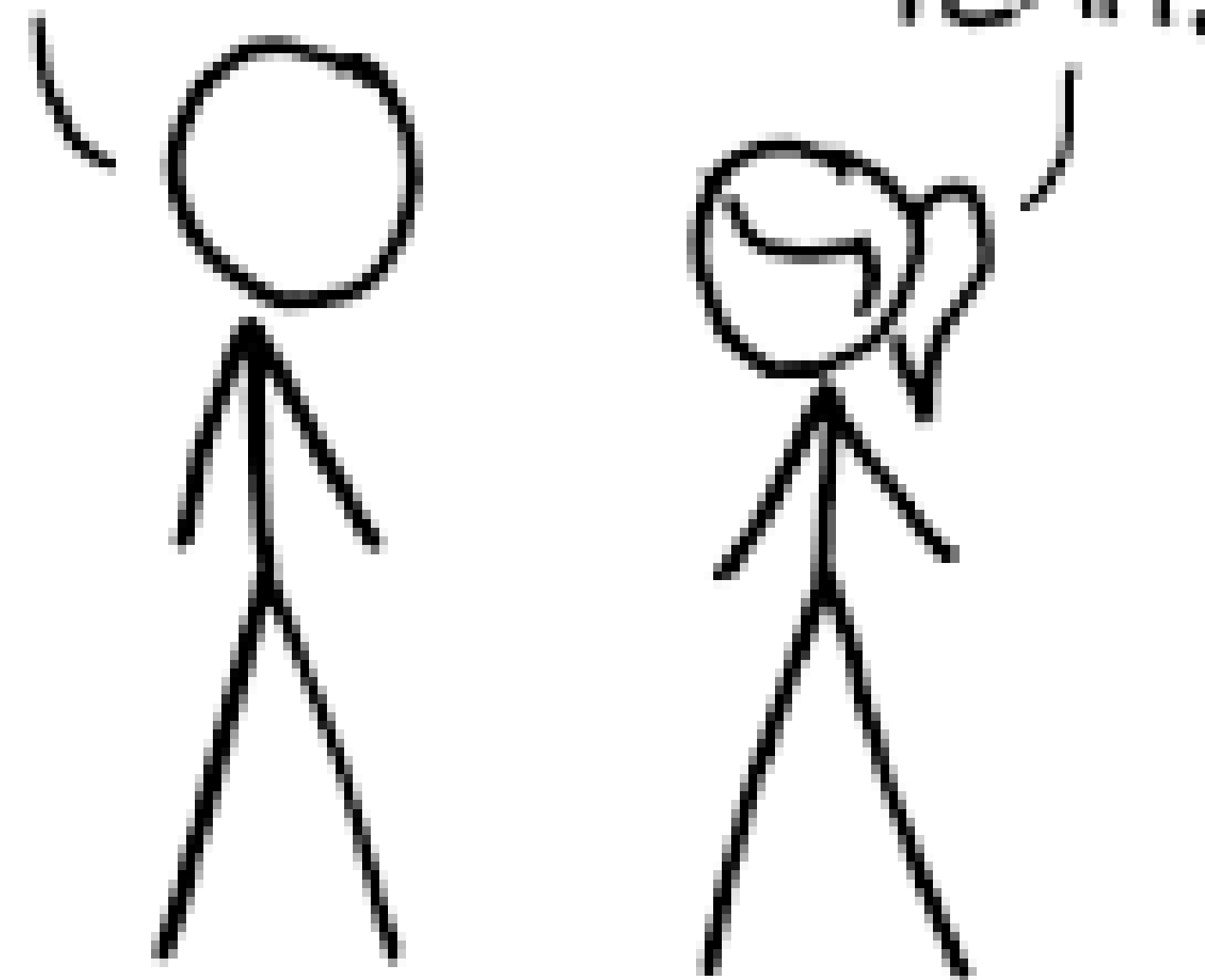
As a user adding a new device or flight model **I can** read a well-written and detailed documentation, **so that** the development process is feasible.

HOW STANDARDS PROLIFERATE:

(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC.)

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



SOON:

SITUATION:
THERE ARE
15 COMPETING
STANDARDS.

Standard selection

Strong requirements:

- Modular architecture, allowing for separate programs, written in different programming languages, running on different computers to participate in the simulation
- Reference implementations and documentation available online for free
- Ability to monitor all exchanged information, for purpose of experimental data collection and debugging

Differentiating factors for ranging acceptable alternatives:

- **Relevant definitions** – Does it come with definitions for information we want to communicate?
 - Information about aircraft like position, attitude etc. in physical units
 - Concept of subsystems of an aircraft
- **Ease of use** – Does it have good documentation? How can it be used with MATLAB? (all of them have C, C++ and Python support)
- **Software constraints** – Does it require a redesign of existing application? Does it impose an execution model like some dynamical co-simulation standards?
- **Wider applicability** – Is it used in the industry? Is it a useful skill to have?

Standard selection

All considered standards (with varying scope):

- [HLA \(High Level Architecture\)](#)
- [DIS \(Distributed Interactive Simulation\)](#)
- **[MAVLink \(Micro Air Vehicle Link\)](#)**
- [FMI \(Functional Mock-up Interface\)](#)
- [DCP \(Distributed Co-Simulation Protocol\)](#)
- **[ZeroMQ](#)**
- [MessagePack](#)
- [Capnproto](#)
- [JSON \(JavaScript Object Notation\)](#)
- [HTTP \(HyperText Transfer Protocol\)](#)
- [WS \(WebSocket\)](#)
- **[MQTT \(Message Queue Telemetry Transport\)](#)**
- **[DDS \(Data Distribution Service\)](#)**
- [ACMI Tacview](#)

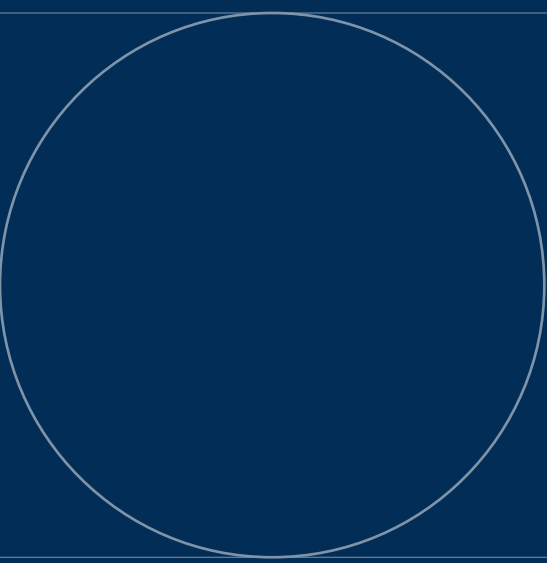
An attempt at a basic prototype was made for the four indicated choices, trying to make a project with:

- MATLAB / Simulink
- Python
- C / C++

These projects and assessment are in the repository:
github.com/marsh-sim/comm-library-comparison

Standard selection

	Relevant Definitions	Ease of use	Software Constraints	Wider Applicability
plain UDP	bad	neutral	good	very bad
HLA	very good	neutral	bad	neutral
DIS	bad	neutral	good	neutral
DDS	neutral	good	neutral	good
MAVLink	good	good	good	good



Framework

03

Overall design

The key insight for the system design was:

Unlike in aircraft systems, we **can** centralise in a simulator.

We don't need redundancy for reliability. We can always stop the test.

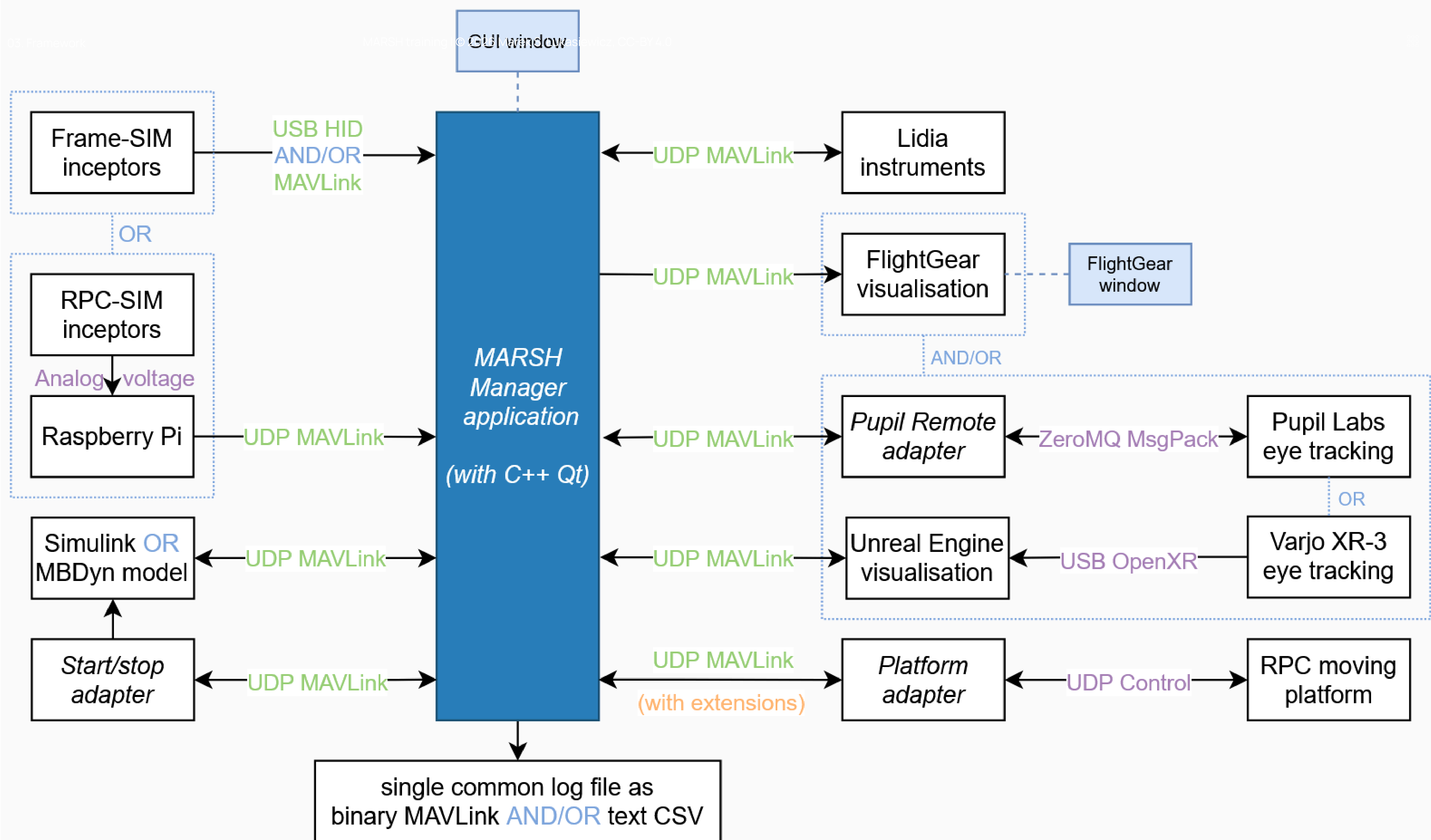
That's one of the reasons we're interested in simulated flight in the first place.

Benefits of centralisation

- Simpler design
- Better compatibility (1-to-N, instead of N-to-N)
- Gather data in a single log
- Single interface for configuration and debugging

Disadvantages

- Performance, but shown to be sufficient in practice



MAVLink protocol



Originally designed for small drones: (Micro Air Vehicles) at ETH Zurich. Inspired by CAN and introduced in 2009, has successfully ensured compatibility between two firmwares that don't share other code: PX4 (under BSD 3-clause) and ArduPilot (under GPL v3), and peripherals for them: sensors, telemetry, control GUI etc.

What we like

- Proven deployment in a well-developed use case
- Lightweight enough to run from MCU over radio
- Comes with many aeronautical definitions
- Extensible via XML definitions
- Libraries for many languages and environments
- Contains a checksum for definition and content
- Messages documented in detail
- Independent of transport (contains framing)

What we dislike

- Sometimes a feature for drones is awkward here
- There are better solutions for lots of data
- Some units are weird to avoid floating point math
- Code is generated from definitions shared a priori
- Libraries are from many authors and differ
- Some implementations are slow (foreshadowing)
- Documentation is often overwhelming
- Adds overhead, some CS purists call it ugly

MAVLink protocol

Even more relevant features

- Separation between:
 1. Telemetry stream (any message) – I just want the latest information, drop the older ones
 2. Reliable delivery (MAV_CMD...) – command must happen, need to confirm it was delivered and accepted
- Configuration interface (parameters)
- Physical units (in message definition)
 - Including quaternion rotations in consistent convention
- Messages are small: binary data with only 12 bytes overhead
 - ...so you can simply save them with timestamps to get a complete log (.tlog)
- Connection and discovery protocol ([HEARTBEAT](#) message)
- Publish-subscribe pattern ([MAV_CMD_SET_MESSAGE_INTERVAL](#))

Some that we don't use (yet?)

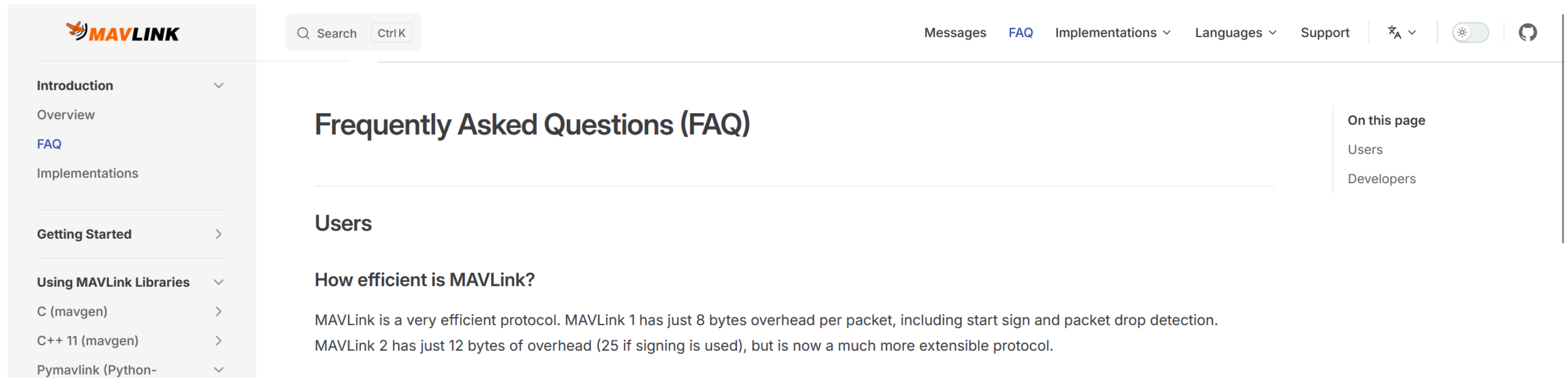
File transfer, peer metadata, clock sync, tunnelling other protocols, cryptographic signing...

MAVLink protocol

Demonstration

Open the documentation at <https://mavlink.io/> and show:

- Introduction, including list of languages
- Serialization
- Common messages
- Dialects



The screenshot shows the MAVLink documentation website. The header includes the MAVLink logo, a search bar with "Search" and "Ctrl K" buttons, and navigation links for "Messages", "FAQ" (highlighted), "Implementations", "Languages", "Support", a language selector, a settings icon, and a GitHub icon. The left sidebar contains a table of contents with sections: "Introduction" (with sub-items "Overview", "FAQ", and "Implementations"), "Getting Started", "Using MAVLink Libraries" (with sub-items "C (mavgen)", "C++ 11 (mavgen)", and "Pymavlink (Python-")

Frequently Asked Questions (FAQ)

Users

How efficient is MAVLink?

MAVLink is a very efficient protocol. MAVLink 1 has just 8 bytes overhead per packet, including start sign and packet drop detection. MAVLink 2 has just 12 bytes of overhead (25 if signing is used), but is now a much more extensible protocol.

On this page

- Users
- Developers

MARSH

Modular Architecture for Reconfigurable Simulation of Helicopters

Has these defining features:

- MAVLink messages and microservices based on them
- UDP transport within one and between many computers
- One manager listening on port 24400 and other nodes

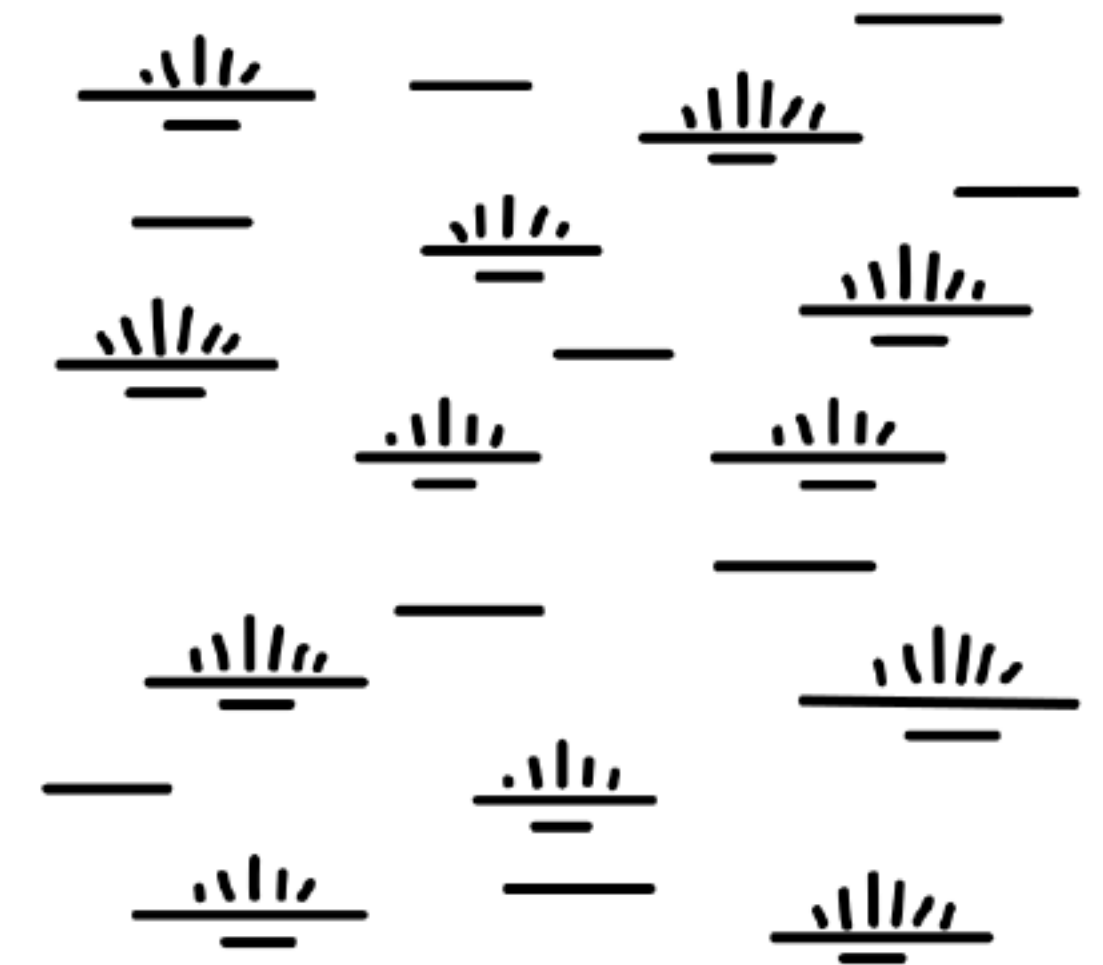
The name is a pun on marshalling (as preparing data for transmission)

Also MAV_ and MARSH_ as a prefix sound pretty similar

Also, marsh is an ecosystem where all the plants and animals are all connected by water.

Also, it's easy to say and doesn't change much between English, Italian and Polish

Also, the name is really not that important



Node registration with Manager

Let us ponder a typical web browser:

1. HTTP server listens on TCP port 80
2. A browser sends HTTP GET request to the server
3. The server replies with page contents on the same network connection
4. The browser asks for more resources or closes the connection

In case of MARSH:

1. MARSH Manager listens on UDP port **24400**
2. A client node sends HEARTBEAT packet from any port and listens for response there
3. Manager registers the client, and sends responses where the initial packet came from
4. Node keeps sending HEARTBEAT packets as long as it's online; when they stop coming, the manager removes the client

Why the differences? We use UDP to discard delayed data, but unlike TCP doesn't have a state itself.

Node registration with Manager

Demonstration

- Start the Manager
- Start `trivial_model.py` from sim-nodes repository

Observe and discuss:

- Starting in different order
- Stopping one or the other
- Multiple copies of node or manager processes
- List of subscribed messages

Show the traffic in Wireshark

Common features

Subscribing to messages

Not every node needs all the possible information. Unnecessary messages cause extra load when sending and receiving, so the messages are filtered by these conditions:

1. Manager sends its [HEARTBEAT](#) to all known nodes
2. Based on node type and autosubscribe list in the manager
3. Requested via [MAV_CMD_SET_MESSAGE_INTERVAL](#)
 - Example for Python: [marsh.py in lidia package](#)
 - Example for C++: [MarshConnector::subscribe_message in Godot visualisation](#)

Or all of the above overridden with [MARSH_MODE_FLAGS](#) in the [HEARTBEAT](#)

The filtering happens in [Router::receiveMessage](#) function inside marsh-manager.

Common features



MARSH Documentation

 / Data analysis / MATLAB  Edit on GitHub

MATLAB analysis

 Note

Requires the "UAV Toolbox " to be installed

The `.tlog` files [saved by MARSH Manager](#) can be loaded with `mavlinktlog`  function. Example code:

```
% This requires a local copy, which would typically be created by update_mavlink.py
dialect = mavlinkdialect("./mavlink_repo/message_definitions/v1.0/marsh.xml");
logimport = mavlinktlog("./data/YYYYMMDDTHHMMSS_comment.tlog", dialect);

sim_state_msgs = readmsg(logimport, 'MessageName', 'SIM_STATE');
sim_state_data = sim_state_msgs.Messages{1};
mps_msgs = readmsg(logimport, 'MessageName', 'MOTION_PLATFORM_STATE');
mps_data = mps_msgs.Messages{1};
```

 Previous

Next 

© 2024, Marek S. Łukasiewicz, CC-BY 4.0

Logging

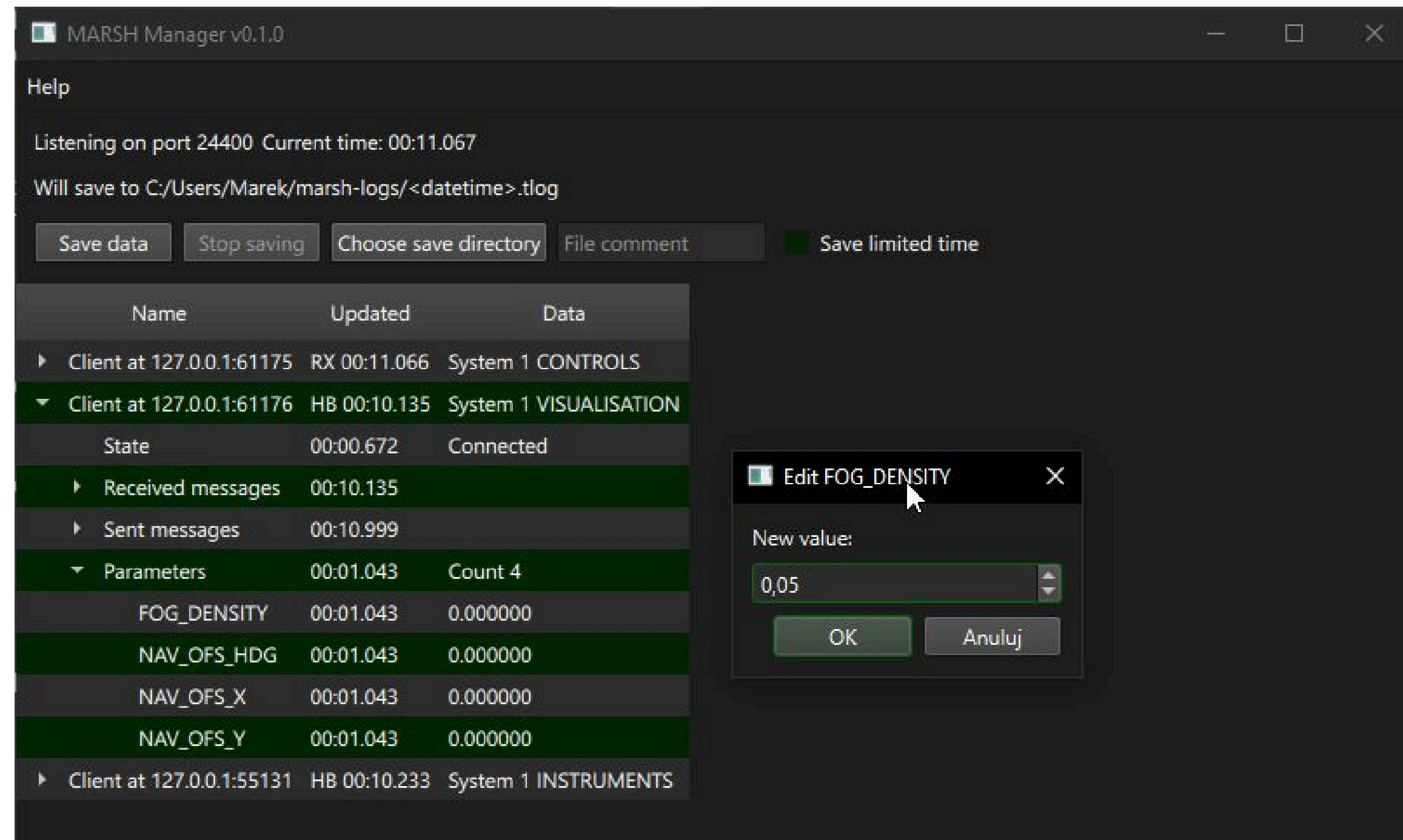
The manager saves all the messages together with their arrival time in a binary “telemetry log” file `.tlog`.

The file is named after start time of saving, and an optional comment. The comment is also saved inside.

The documentation contains code examples for log analysis in MATLAB and Python:

- <https://marsh-sim.github.io/analysis/matlab/>
- <https://marsh-sim.github.io/analysis/python/>

Common features



Parameters

Each node can provide a set of configurable values as parameters. The names must be 16 bytes or shorter, by convention UPPER_CASE ASCII is used. The values are 32-bit floats.

The manager automatically asks each new node about its parameters with [PARAM_REQUEST_LIST](#). The values can be seen and changed in the GUI.

See MAVLink documentation for [Parameter Protocol](#), for detailed instructions how to do it through code.

Timing

By design, various components do **not** run in lockstep. Each just uses latest received data.

This is a compromise that slows information flow, and adds non-deterministic network delays. However, it makes the nodes a lot easier to develop, and enables more languages and frameworks to be used.

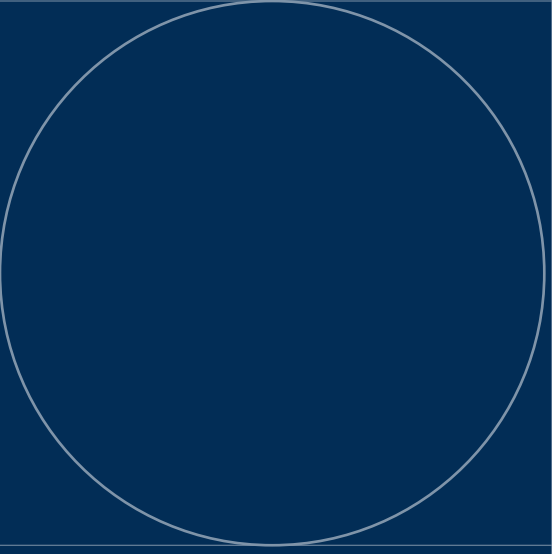
With programs located on the same computer, localhost networking is sub-millisecond, but it gives the flexibility to move them to other hosts when needed (unlike shared library approach).

Publications about **microservices** deal with these questions, it is a well-known topic (debate?).

Improving performance

There are still many low hanging fruits to reach for to improve performance:

- Rewrite Python programs in a high-performance language
- Replace hardware adapter scripts with the firmware directly using MAVLink messages
- Tune the performance of Manager, so far it's just plain Qt event loop
- Send messages through Unix domain sockets instead of network when on same machine



Simulator nodes

04

Cutting up the helicopter

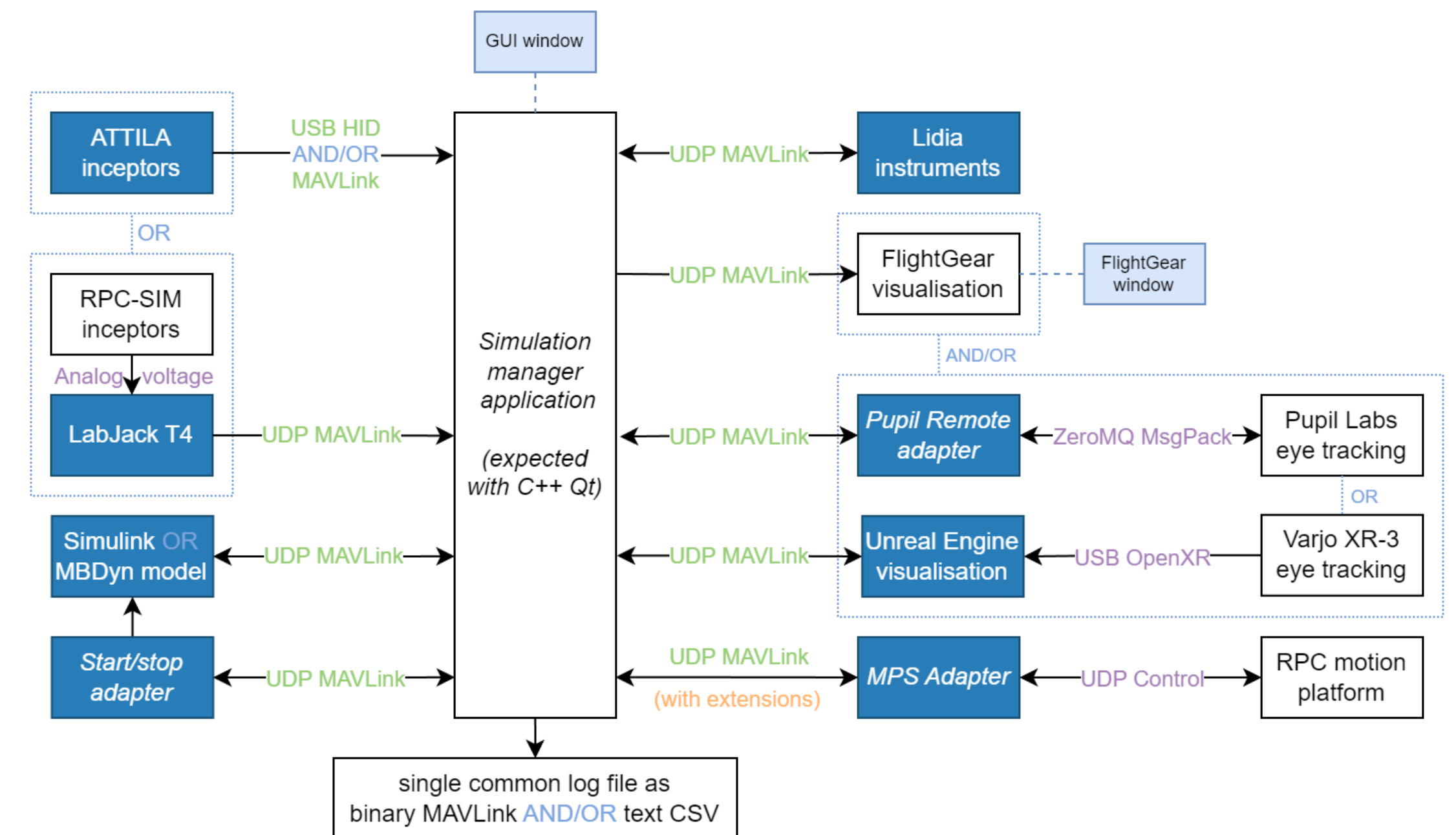
Each node has a single responsibility, which makes them easier to develop, test and reuse.

On the other hand, keeping them separate is worse for performance, and they need to be started separately.

For a basic helicopter flight, you need to run at least:

- Manager
- Controls
- Flight model
- Visualisation or Instruments

The only thing to configure is the manager's address, so all nodes accept `-m` or `--manager` argument to pass that through command line.



Controls

Measures inceptor positions from hardware and sends [MANUAL_CONTROL](#). Also handles buttons.

If a separate flight control system node is used, controls should send [MANUAL_SETPOINT](#) instead.

Nodes developed so far are in the [sim-nodes repository](#):

- For any USB HID device [joystick_controls.py](#)
 - Axis assignment and directions can be modified with parameters
 - Presets for our devices built in
 - Handles all buttons
- For Labjack T4 device [labjack_controls.py](#):
 - Measures positions with analog sensors using Labjack's ADC
 - Voltages for axis end positions can be calibrated with parameters
 - Implements collective with thumbstick mixed in by Paolo de Franceschi

Flight models

Subscribes [MANUAL_CONTROL](#), does the simulation of flight mechanics and sends [SIM_STATE](#) and/or: [ATTITUDE](#), [LOCAL_POSITION_NED](#), [RAW_RPM](#) and any other message if more details are simulated.

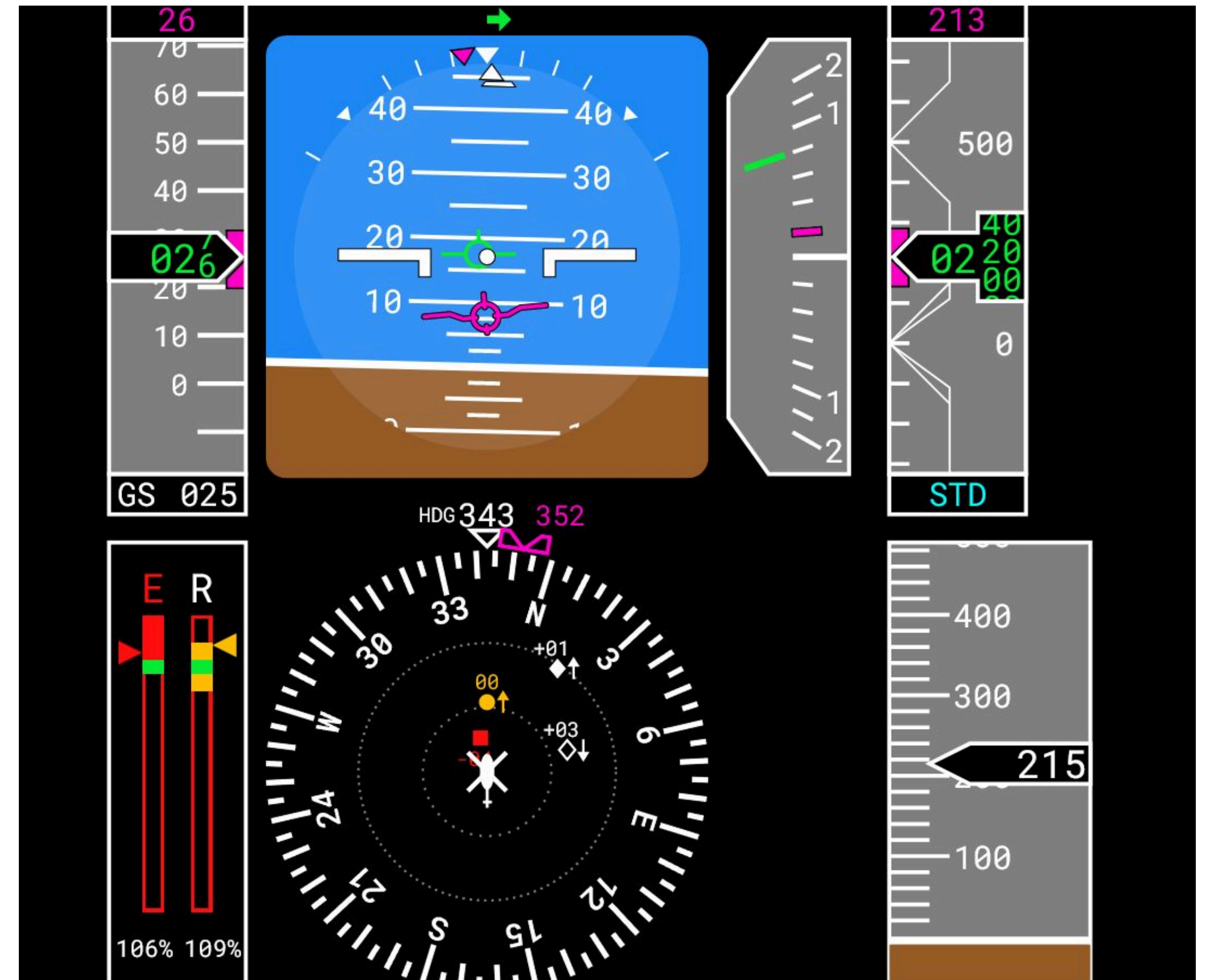
- Linear Time-Invariant Simulink models in [helisim-lti repository](#), **on the marsh-adapter branch**.
 - based on “Helicopter Flight Dynamics: Including a Treatment of Tiltrotor Aircraft” book by Gareth Padfield
 - these are models with 6 degrees of freedom, modified to use attitude from quaternion integration
 - identified from flight data for Bo-105, Lynx and Puma helicopters, with optional stability augmentation
- Collective bounce 2 DoF Simulink model in [cb2dofCL repository](#) **on the marsh branch**
 - based on “The Role of Rotor Coning in Helicopter Proneness to Collective Bounce” by Muscarello, V. & Quaranta, G. & Masarati P.
 - only moves vertically with ground contact, has an internal degree of freedom for rotor cone angle
 - tends to oscillate quite strongly, which can be adjusted with collective to blade pitch gear ratio
- Trivial tilt Python model in [sim-nodes repository](#)
 - direct passthrough of commands, mostly made for testing other components

Instruments

Meant to display information to the pilot and/or simulator operator. Subscribes [MANUAL_CONTROL](#), [SIM_STATE](#), and typically requests more, depending on the features.

Currently, all screens are implemented in a Python package that serves its views in a web browser. You can install it with:

```
pip install lidia
```

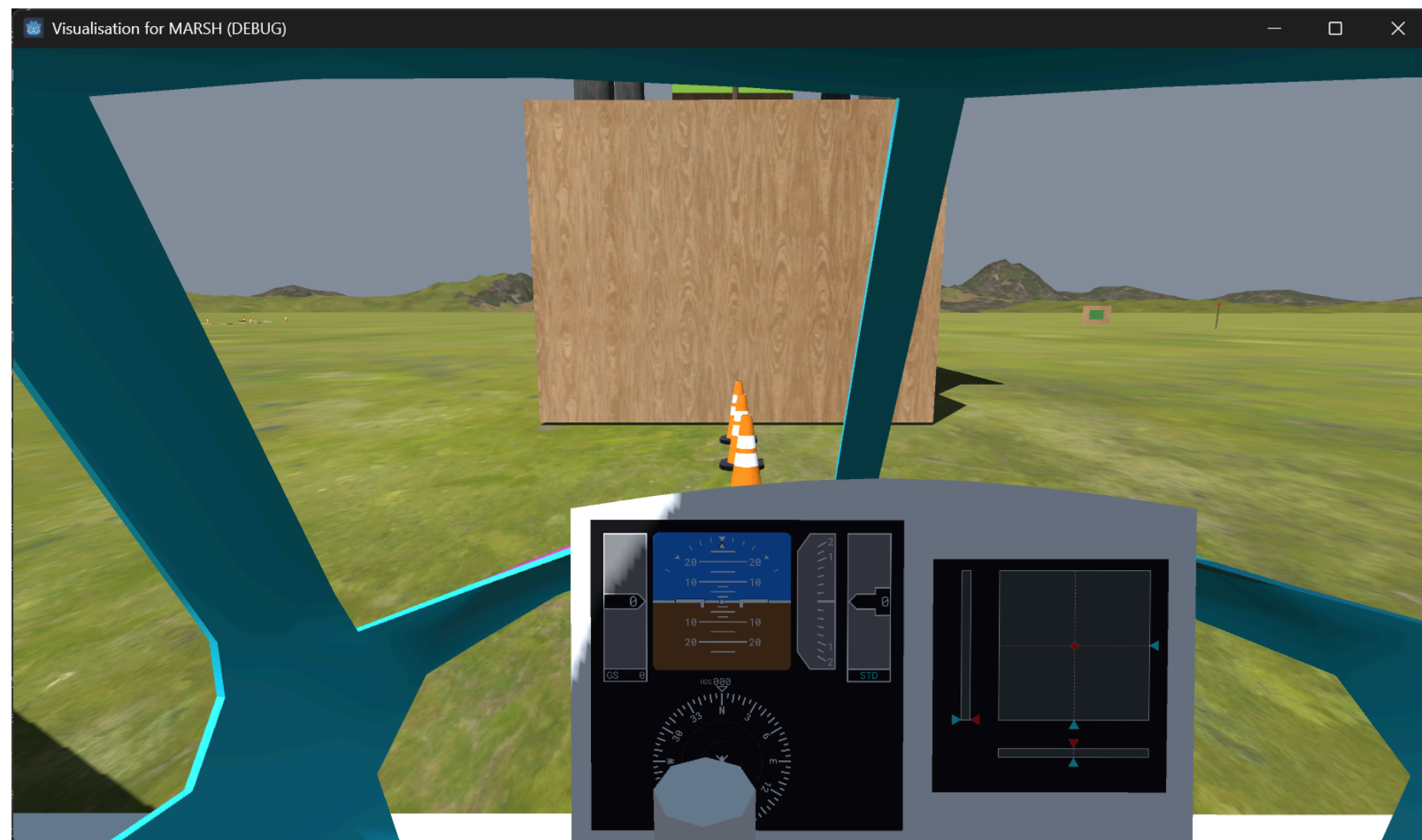


Visualisations

Subscribes `SIM_STATE` and other messages about the aircraft state and shows it to the pilot. Since it doesn't send anything, you can have many of them without any conflict.

Currently, only the left visualisation made with Godot Engine is supported, hosted on:

- <https://gitlab.polimi.it/DAER/frame-sim/marsh/visualisation-marsh>
- <https://git.lukasiewicz.tech/DAER/visualisation-marsh> (publicly accessible mirror)

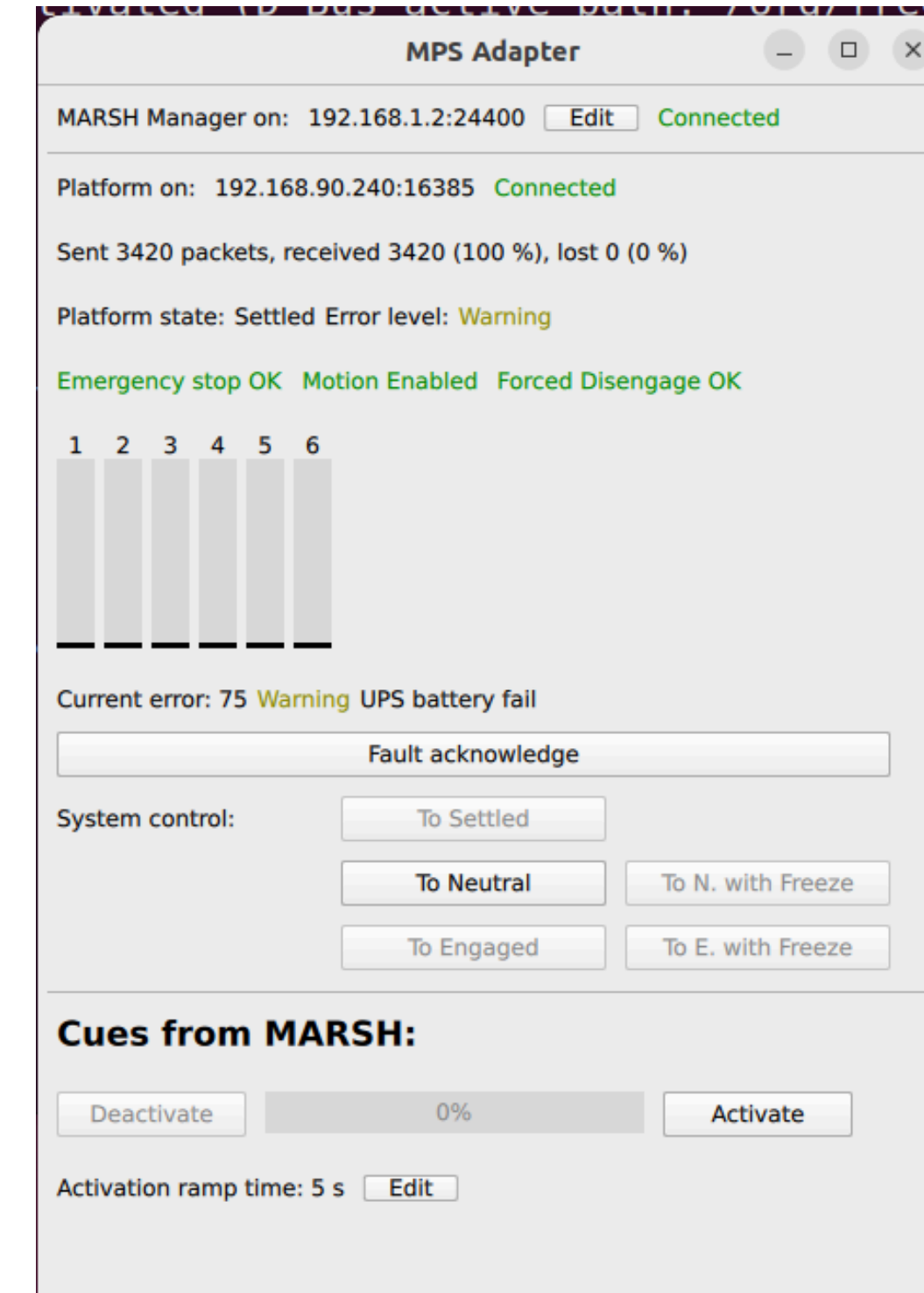


Motion cue

Subscribes [SIM_STATE](#), [HIGHRES_IMU](#) and [MOTION_CUE_EXTRA](#) to provide motion feeling to the pilot. These report their state with [MOTION_PLATFORM_STATE](#) and optionally [REXROTH_MOTION_PLATFORM](#).

So far two have been implemented:

- Motion Platform System adapter with GUI
 - ▶ The repository [mps-adapter](#) is not public, because it contains Bosch Rexroth proprietary messaging format.
 - ▶ Controls the platform, and enforces limits
 - ▶ Physical units verified at compile time
 - ▶ Unit tests for safety code
- G-Seat controller ([sim-nodes/vertical_uart_gseat.py](#))
 - ▶ Developed for a specific device in ATTILA simulator
 - ▶ This is the “Haptic Device Embedded in Rotorcraft Seat to Provide Motion Onset Cues in Flight Simulator”



Extra tools

Other experiment scenarios

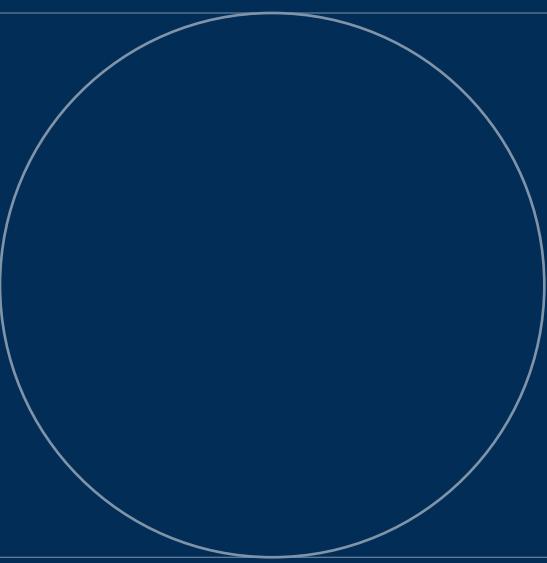
- [Cue identification](#) – GUI to do frequency sweep with different limits for motion cues above
- [Platform test](#) – interactive console app with plots to validate correct behaviour of motion platform
- [Extra waveform](#) – reproducible pseudorandom noise generator for a configurable frequency band

More biometric data

These are developed so that data from the device is saved synchronously in the same .tlog file.

- [Eye tracking](#) – integrate data from Pupil eye tracking glasses
- [Body pose](#) – pilot's body position tracked from a 3D camera feed
- EMG – electromyography, measures muscle activation





Development

05

New sim nodes

Advice for making a new node:

- Consider the data it will read and provide, and assign messages to that
- See if it matches with an existing type, otherwise define a new one in the [marsh.xml file](#)
- Choose your preferred programming environment
- Test early, verify in Manager that you're detected, and sending correct values
- Expose configuration as parameters, even if it cannot change at runtime (just don't change anything when receiving [PARAM_VALUE](#))
- Make it possible to pass the IP address of Manager, preferably via `-m` and `--manager` command line argument.

Types we have defined so far in [MARSH_TYPE](#):

- Manager
- Flight Model
- Controls
- Visualisation
- Instruments
- Motion Platform
- Gseat
- Eye Tracker
- Control Loading
- Vibration Source
- Pilot Target
- Experiment Director
- EMG
- Body Tracker

Exchanging data

See if there are already features describing the data you need. In addition to [MARSH documentation](#), see also the full [MAVLink Common Message Set](#), and [Microservices](#) list.

For one-off usage and prototyping, consider [NAMED_VALUE_FLOAT](#), [V2_EXTENSION](#) and [STATUSTEXT](#).

One-off events that need to be reliably delivered should be [Commands](#). They can have a 3D position and four 32-bit float parameters, example [MAV_CMD_DO_EXPERIMENT_CONTROL](#).

Use [named enumerations \(enums\)](#) to identify states, types, points, sensors ([see examples](#)).

Avoid mistakes by using units in field definitions ([list of accepted units](#)), and if the specific unit is not defined in the schema, specify it in the description. Clarify value ranges and directions.

Use a text editor that checks XML syntax, and can verify it against a schema document (for example Visual Studio Code). To get the `mavschema.xsd` in the right path, make sure you have the `pymavlink` submodule checked out: `git submodule update --init`

Exchanging data

Enum example

```
<enum name="CONTROL_AXIS">
  <description>Specific axis of pilot control inputs, with order corresponding to x, y, z, r fields in MANUAL_CONTROL message.</description>
  <entry value="0" name="CONTROL_AXIS_ROLL">
    <description>Roll axis, with positive values corresponding to stick right movement, causing the vehicle to roll right. For helicopters this is lateral cyclic.</description>
  </entry>
  <entry value="1" name="CONTROL_AXIS_PITCH">
    <description>Pitch axis, with positive values corresponding to stick forward movement, causing the vehicle to move nose down. For helicopters this is longitudinal cyclic.</description>
  </entry>
  <entry value="2" name="CONTROL_AXIS_THRUST">
    <description>Main thrust, with positive values corresponding to going faster and higher. For helicopters this is collective.</description>
  </entry>
  <entry value="3" name="CONTROL_AXIS_YAW">
    <description>Yaw axis, with positive values corresponding to pushing right pedal, causing the vehicle to face right direction. For helicopters this is tail collective.</description>
  </entry>
</enum>
```

Exchanging data

Command example

```
<entry value="52500" name="MAV_CMD_DO_EXPERIMENT_CONTROL">  
  <description>Control experiment data collection. Used to synchronize start/stop of data recording  
across multiple simulation components. Parameters 2 and 3 can be used by applications to match the name  
of log file created by MARSH Manager.</description>  
  <param index="1" label="Enable">Enable (1) or disable (0) experiment data collection.</param>  
  <param index="2" label="Log Day">Date of logging start, stored as days since 1970-01-01. Quotient of  
Unix epoch divided by 86400.</param>  
  <param index="3" label="Log Seconds">Time of logging start, stored as seconds since UTC midnight.  
Remainder of Unix epoch divided by 86400.</param>  
  <param index="4">Reserved (set to 0)</param>  
  <param index="5">Reserved (set to 0)</param>  
  <param index="6">Reserved (set to 0)</param>  
  <param index="7">Reserved (set to 0)</param>  
</entry>
```

Exchanging data

Message example

```
<message id="52501" name="CONTROL_LOADING_AXIS">
  <wip/>
  <!-- This message is work-in-progress and it can therefore change. It should NOT be used in stable
production environments. -->
  <description>Send data about a control axis from a control loading system. This is the primary message
for logging data from MARSH_TYPE_CONTROL_LOADING.</description>
  <field type="uint32_t" name="time_boot_ms" units="ms">Timestamp (time since system boot).</field>
  <field type="uint8_t" name="axis" enum="CONTROL_AXIS" instance="true">Control axis on which the
measurements were taken.</field>
  <field type="float" name="position" units="deg">Axis position</field>
  <field type="float" name="velocity" units="deg/s">Axis velocity</field>
  <field type="float" name="force">Force applied in the pilot in the direction of movement axis (not
gripping force), measured at the position of pilot's third finger (ring). Unit N (Newton), currently not
part of mavschema.xsd</field>
</message>
```

Python

```
#!/usr/bin/env python3
```

```
"""
```

```
Main parts of a simulator node
```

```
"""
```

```
from argparse import ArgumentParser
```

```
import mavlink_all as mavlink
```

```
from pymavlink import mavutil
```

```
from time import time
```

```
parser = ArgumentParser(description=__doc__)
```

```
parser.add_argument('-m', '--manager', help='MARSH Manager IP addr', default='127.0.0.1')
```

```
args = parser.parse_args()
```

```
mav = mavlink.MAVLink(mavutil.mavlink_connection(f'udpout:{args_manager}:24400'))
```

```
mav.srcSystem = 1 # default system
```

```
mav.srcComponent = mavlink.MAV_COMP_ID_USER1 + (mavlink.MARSH_TYPE_FLIGHT_MODEL - mavlink.MARSH_TYPE_MANAGER)
```

```
# continuing...
```

Python

```
# continued
heartbeat_next = time()
while True:
    if time() >= heartbeat_next:
        mav.heartbeat_send(
            mavlink.MARSH_TYPE_FLIGHT_MODEL, mavlink.MAV_AUTOPILOT_INVALID, mavlink.MAV_MODE_FLAG_TEST_ENABLED, 0,
            mavlink.MAV_STATE_ACTIVE)
        heartbeat_next = time() + 1.0

    try:
        while (message := mav.file.recv_msg()) is not None:
            message: mavlink.MAVLink_message
            if message.get_type() == 'HEARTBEAT':
                if message.type == mavlink.MARSH_TYPE_MANAGER:
                    if not manager_connected:
                        print('Connected to simulation manager')
                        manager_connected = True
                    manager_timeout = time() + timeout_interval
            elif message.get_type() == 'MANUAL_CONTROL':
                last_controls = Controls.from_message(message)
    except ConnectionResetError:
        pass # thrown on Windows when there is no peer listening
```

MATLAB

Not for real-time running, reproduced from <https://marsh-sim.github.io/analysis/matlab/>

```
% This requires a local copy, which would typically be created by update_mavlink.py
dialect = mavlinkdialect("./mavlink_repo/message_definitions/v1.0/marsh.xml");
logimport = mavlinktlog("./data/YYYYMMDDTHHMMSS_comment.tlog", dialect);

sim_state_msgs = readmsg(logimport, 'MessageName', 'SIM_STATE');
sim_state_data = sim_state_msgs.Messages{1};
mps_msgs = readmsg(logimport, 'MessageName', 'MOTION_PLATFORM_STATE');
mps_data = mps_msgs.Messages{1};
```

C and C++

In practice, most implementors use the C library, even in C++ codebases

```
#include "mavlink/all/mavlink.h"
```

```
uint8_t udp_data[MAVLINK_MAX_PACKET_LEN]; // somehow filled with the UDP packet contents
mavlink_message_t message;
mavlink_status_t parser_status;
```

```
for (size_t i = 0; i < sizeof(udp_data); i++) {
    if (mavlink_parse_char(MAVLINK_COMM_0, udp_data[i], &message, &parser_status)) {
        switch (message.msgid) {
            case MAVLINK_MSG_ID_HEARTBEAT:
                handle_heartbeat(message);
                break;
            case MAVLINK_MSG_ID_SIM_STATE:
                handle_sim_state(message);
                break;
            default:
                break;
        }
    }
}
```

C and C++

```
void MarshConnector::handle_heartbeat(mavlink_message_t message) {  
    mavlink_heartbeat_t heartbeat;  
    mavlink_msg_heartbeat_decode(&message, &heartbeat);  
    if (heartbeat.type == MARSH_TYPE_MANAGER) {  
        if (!manager_connected) {  
            std::cout << "Connected to MARSH Manager";  
            manager_connected = true;  
        }  
    }  
}
```

Simulink

- Ensure local port is -1 to get a random one
- Receiving has acceptable performance, sending is very slow, because it does the CRC in MATLAB code
 - Because of that, we gather all inputs and encode them using a C function
- Tends to be very slowed down by many inputs, this is where we use [MARSH_MODE_SINGLE_MESSAGE](#).

The advice above is sufficient to run the `helisim-lti` model at 100 Hz in regular mode. Using the Real-Time or Code Generator packages correctly, more features can probably be implemented.

Demonstration

Just show the `helisim-lti` model, too many gotchas to document them here.